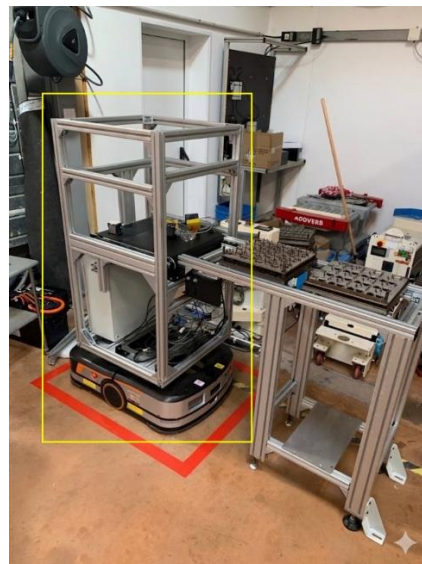
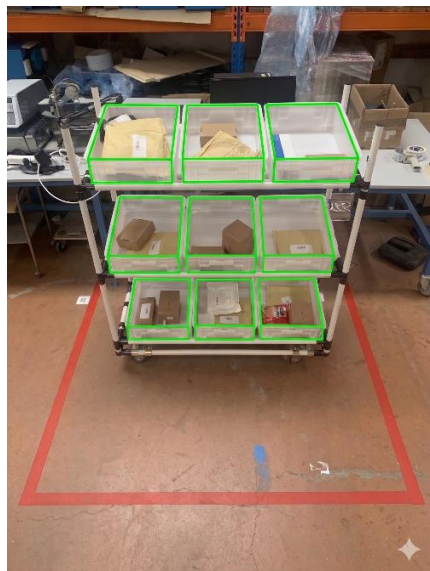
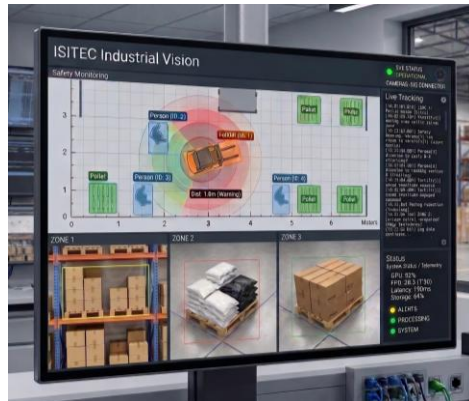


# Cahier des Charges

## Système de Vision Industrielle



|                                            |   |
|--------------------------------------------|---|
| ○ Introduction & État des Lieux            | 2 |
| ○ Architecture du Backbone                 | 2 |
| ○ Exigences Fonctionnelles (Modular Ports) | 3 |
| ○ Performance et Standards                 | 4 |
| ○ Plan de Projet (Sprint de 3 mois)        | 4 |
| ○ Spécifications des Coûts d'Inférence     | 6 |

# 1. Introduction & État des Lieux

## 1.1 Contexte Actuel

Le système actuel génère une vue 2D simplifiée de la réalité à partir des caméras, en projetant des icônes 3D d'objets (palettes, chariots, personnes) sur un plan au sol. Cela permet de comprendre rapidement l'activité en un coup d'œil.

## 1.2 La Problématique

Bien que visuellement clair, le système actuel repose sur des heuristiques d'espace image (u,v) plutôt que sur une réalité physique. Cette approche présente des limites majeures pour une utilisation industrielle :

- **Absence de Raisonnement Métrique** : fonctionne en pixels, non en distances réelles (mètres), rendant les alertes de sécurité imprécises.
- **Instabilité Géométrique** : Toute modification de l'angle ou de la position de la caméra brise instantanément la cohérence du mapping.
- **Vulnérabilité aux Occlusions** : La dépendance à une vue unique rend les objets "invisibles" dès qu'ils passent derrière un obstacle.
- **Incapacité à prendre des décisions**: reste un outil de visualisation, incapable d'arrêter une machine ou de valider un inventaire.

## 1.3 Objectif du Nouveau Système

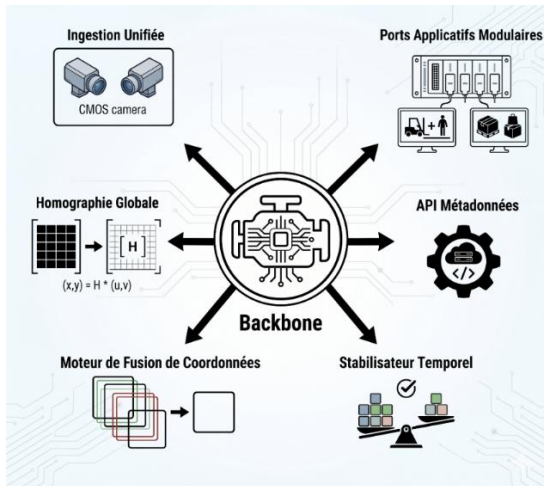
L'objectif est de passer d'un simple afficheur à une plateforme de décision industrielle. En introduisant un Backbone spatial métrique (localisation en temps réel en mètres), le système devient capable de fusionner plusieurs caméras pour créer un "Digital Twin" géométriquement exact, robuste et modulaire.

## 2. Architecture du Backbone – Le socle commun

Le Backbone est le moteur central qui transforme des flux vidéo bruts en données exploitables. Il traite simultanément les flux vidéo de plusieurs caméras CMOS via RTSP, convertit les pixels en coordonnées réelles (x, y en mètres) sur un plan unique par homographie, puis fusionne les détections des vues qui se chevauchent pour lever les occlusions et garantir une position unique par objet.

Un stabilisateur temporel analyse l'historique des trames (vote majoritaire) pour éviter les faux positifs.

Enfin, le Backbone expose les métadonnées (ID, classe, position x,y) via une API, permettant aux modules métier (Sécurité, Palettes, etc.) de fonctionner de manière indépendante et plug-and-play.



### 3. Exigences fonctionnelles – Les modules indépendants

Le système est conçu de manière modulaire : des modules "ports" s'ajoutent ou se mettent à jour sans toucher au Backbone ; une pratique industrielle standard où chaque partie peut être diagnostiquée et mise à jour sans affecter ni bloquer les autres.

**Module Sécurité** : Suit les personnes et chariots, calcule en temps réel les distances métriques entre eux, et déclenche des alertes visuelles ou API en cas de violation de périmètre. Il transforme les caméras en capteurs anticollision.

**Module Palettes & Inventaire** : Localise les palettes (YOLO), les classe "Vide" ou "Chargée" (MobileNet), puis les affecte à des zones de stockage (point-in-polygon). Automatise la mise à jour des stocks sans intervention humaine.

**Module Statut des Rayonnages** : Découpe l'image en alvéoles de racks (ROI), détecte si chaque emplacement est vide ou occupé, et transmet l'information en temps réel. Étend la vision 2D au sol à la gestion du stockage en hauteur

## 4. Performance et Standards (Attendus)

### 4.1 Indicateurs clés (KPI)

Pour être fiable, le système fonctionne en autonomie totale, sans dépendre du cloud et doit respecter ces seuils :

| Indicateur                 | Seuil           | Objectif                               |
|----------------------------|-----------------|----------------------------------------|
| Latence globale            | < 200 ms        | Du flux vidéo à l'événement            |
| Précision géométrique      | $\leq 2$ pixels | Garantir des mesures métriques valides |
| Détection (mAP@0.5)        | $\geq 0,90$     | Localiser fiablement les objets        |
| Classification (précision) | $\geq 0,95$     | Distinguer palette vide / pleine       |
| Classification (rappel)    | $\geq 0,93$     | Ne pas manquer de palettes             |

### 4.2 Déploiement et Validation

- Recalibrage obligatoire de l'homographie si une caméra est déplacée physiquement.
- Les zones et seuils sont modifiables via fichiers de configuration, sans toucher au code.
- Code source complet, modèles entraînés, documentation UI et tutoriel.

## 5. Plan de projet – Sprint de 3 mois

### Mois 1 : Construction du moteur principal (Backbone)

|              |                                                                                                                                                   |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Semaines 1–2 | Configuration des caméras (flux vidéo RTSP) et mise en place de la matrice d'homographie pour convertir les images en coordonnées réelles au sol. |
| Semaine 3    | Fusion des données de plusieurs caméras pour créer une seule « source de vérité » (gère les occlusions).                                          |
| Semaine 4    | Installation d'un système de messagerie pour que le Backbone envoie les données aux modules sans être modifié ensuite.                            |

## Mois 2 : Détection et modules intelligents

|              |                                                                                                                                                                                   |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Semaines 5–6 | Entraînement du modèle YOLO pour détecter les personnes, chariots élévateurs et palettes.                                                                                         |
| Semaine 7    | Développement en parallèle <ul style="list-style-type: none"><li>• Alertes de sécurité (proximité dangereuse)</li><li>• Classifieur MobileNet « palette vide / pleine »</li></ul> |
| Semaine 8    | Surveillance des zones de stockage (rayonnages) + filtrage temporel pour stabiliser les détections                                                                                |

## Mois 3 : Interface utilisateur, tests et livraison

|               |                                                                                                                                                     |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Semaines 9–10 | Création d'un tableau de bord web (vue 3x3 des caméras + carte 2D) et d'une API (JSON/UDP) pour communiquer avec les systèmes industriels (WMS/PLC) |
| Semaine 11    | Test de charge – garantir une latence totale < 200 ms sur le matériel NVIDIA Jetson Orin NX.                                                        |
| Semaine 12    | Livraison du code source, des modèles entraînés et de la documentation complète.                                                                    |

### 5.2. Liste des livrables finaux

| Livrable                     | Contenu technique                                                     |
|------------------------------|-----------------------------------------------------------------------|
| Moteur Backbone              | Homographie, fusion de coordonnées, serveur de messages               |
| Modèles                      | YOLO (détection personnes/chariots/palettes) + MobileNet (vide/plein) |
| Interface (jumeau numérique) | Carte 2D globale + vue grille 3x3 des caméras                         |
| Passerelle API               | Scripts de communication JSON/UDP pour WMS et automates (PLC)         |
| Documentation                | Guide d'installation et de calibration des caméras                    |

## 6. Spécifications des Coûts d'Inférence

Ce budget 'Quick Setup' est basé sur des composants sourcés en France, garantissant une compatibilité maximale avec le reComputer J4012 et une robustesse adaptée au milieu industriel.

### Devis par Nœud (Configuration 2 Caméras)

| Composant           | Qté | Référence Technique                                    | Coût Unit. | Sous-total     |
|---------------------|-----|--------------------------------------------------------|------------|----------------|
| Edge AI             | 1   | Seeed reComputer J4012 (Orin NX 16GB)                  | 1 080 €    | 1 080 €        |
| SSD Interne         | 1   | Seeed Studio 256GB NVMe M.2 PCIe Gen3x4                | 69 €       | 69 €           |
| Caméras 4K          | 2   | Hikvision ColorVu DS-2CD2387G2H-LIU                    | 230 €      | 460 €          |
| Switch PoE (France) | 1   | Teltonika TSW200 (Industrial PoE+ 8-Port)              | 85 €       | 85 €           |
| Infrastructure      | –   | Câblage Cat6 blindé, fixations et boîtiers de jonction | 150 €      | 150 €          |
| <b>TOTAL</b>        | 1   | Investissement Nœud Isitec (2 caméras)                 |            | <b>1 844 €</b> |



### 6.1 Justification Économique et Technique

- **Switch Teltonika TSW200** (85 €) : Meilleur rapport qualité/prix industriel. Boîtier aluminium et PoE+ (30W/port) essentiels pour alimenter les LEDs des caméras ColorVu sans risque de surcharge.
- **Caméra ColorVu** (230 €) : Capteur 8MP garantissant une image couleur 24h/24, réduisant les faux positifs de détection de près de 40 % par rapport à l'infrarouge classique.
- **Écosystème Seeed Studio** : SSD officiel (69 €) + module J4012 (1 080 €) permettent un boot direct sur NVMe, assurant une installation sans erreur et une interface de contrôle réactive.